

(1)

	A	B	C
put	$O(1)$	$O(n)$	$O(\log n)$
get	$O(n)$	$O(1)$	$O(\log n)$

解説:A の put は最後の要素の次に挿入するだけなので $O(1)$,同様に B の get も最後の要素を削除するだけなので $O(1)$.A の get は最悪の場合すべての要素を探索しないといけないので $O(n)$,同様に B の put も最悪の場合すべての要素を探索しないといけないので $O(n)$.
C の put と get は最悪の場合二分木ヒープソートの高さの分探索しないといけないのでともに $O(\log n)$.

(2)

	A	B	C
put	$O(n)$	$O(n^2)$	$O(n \log n)$
get	$O(n^2)$	$O(n)$	$O(n \log n)$

解説:A の put と B の get は n 回挿入あるいは削除するだけなので $O(n)$.A の get と B の put は操作ごとに要素の数だけ計算が必要なので,

$$\begin{aligned}
 & O(1 + 2 + \dots + n) \\
 &= O\left(\sum_{i=0}^n i\right) \\
 &= O\left(\frac{1}{2}n(n-1)\right) \\
 &= O(n^2)
 \end{aligned}$$

となる.C については各操作ごとに(1)で求めたオーダーの分計算が必要になるので,

$$\begin{aligned}
 & O(\log 1 + \log 2 + \dots + \log n) \\
 &= O\left(\sum_{i=0}^n \log n\right) \\
 &= O(\log n!) \\
 &= O(n \log n)
 \end{aligned}$$

となる.

(3)get() $=3$,size $=6$,x $=\{3,4,6,9,7,8,8,0,0\}$

解説:順に実行していくと以下ようになる.

```
C:\WINDOWS\system32\cmd.exe

  7
0 0
0 0 0 0
0 0

-----

  3
  7 0
0 0 0 0
0 0

-----

  1
  7 3
0 0 0 0
0 0

-----

  1
  7 3
9 0 0 0
0 0

-----

  1
  3 3
9 7 0 0
0 0

-----
```

```
C:\WINDOWS\system32\cmd.exe

get()=1

  3
  7 3
9 7 0 0
0 0

-----

  3
  4 3
9 7 0 0
0 0

-----

  3
  4 3
9 7 6 0
0 0

-----

  3
  4 3
9 7 6 8
0 0

get()=3

  3
  4 6
9 7 8 8
0 0
```

```
選択C:\WINDOWS\system32\cmd.exe

0 0

get()=3

  3
  4 6
9 7 8 8
0 0

-----

x=3,4,6,9,7,8,8,0,0.
size = 6
続行するには何かキーを押してください . . .
```

(4)

```
void up(int i){
    if(i<=0) return;
    int p = (i-1)/2;
    while(x[p]>x[i]){
        swap(p,i);
        i= p;
        p = (p-1)/2;
    }
}
```

(5)

```
void heapsort(int n) {
    for (int i = 0; i < n-1; i++) {
        x[size] = get();
    }
}
```

解説:以下に実行内容を示す.(ただし n=size,問題設定は(3)と同様のものとして実行している)

操作終了後

配列番号	0	1	2	3	4	5	6	7	8
中身の値	3	4	6	9	7	8	8	0	0

for1 回目

配列番号	0	1	2	3	4	5	6	7	8
中身の値	4	7	6	9	7	3	8	0	0

for2 回目

配列番号	0	1	2	3	4	5	6	7	8
中身の値	6	7	8	9	4	3	8	0	0

for3 回目

配列番号	0	1	2	3	4	5	6	7	8
中身の値	7	9	8	6	4	3	8	0	0

省略

ラスト

配列番号	0	1	2	3	4	5	6	7	8
中身の値	9	8	7	6	4	3	8	0	0

このように降順に並べ替えることができる.x[6]=8 があるが今回並び替えをしたい対象ではないのでそのままである.

付録

最後にこの問題の解答作成のために作成したソースコードを以下に示す.各自確認のため使用してもらいたい.

```
#include <stdio.h>
#define MAXSIZE 9

int x[MAXSIZE];
int size;

void swap(int i, int j) {
    int t;
    t = x[i];
    x[i] = x[j];
    x[j] = t;
}

void up(int i) {
    if (i <= 0) return;
    int p = (i - 1) / 2;
    //if (x[p] > x[i]) {
    while(x[p]>x[i]){
        swap(p, i);
        //up(p);
        i = p;
        p = (p - 1) / 2;
    }
}

void down(int i, int m) {
    int c = 2 * i + 1;
    if (c >= m) return;
    if ((c + 1) < size && (x[c + 1] < x[c])) {
        c++;
    }
}
```

```

    }
    if (x[i] > x[c]) {
        swap(c, i);
        down(c, m);
    }
}

void put(int value) {
    if (size >= MAXSIZE) return;
    x[size] = value;
    size++;
    up(size - 1);
}

int get(){
    if(size <= 0) return 0;
    int result;
    result = x[0];
    x[0] = x[--size];
    down(0, size);
    return result;
}

void heapsort(int n) {
    for (int i = 0; i < n; i++) {
        x[size] = get();
    }
}

void output() {
    printf("-----¥n");
    printf("    %d    ¥n", x[0]);
    printf("  %d    %d  ¥n", x[1], x[2]);
    printf(" %d %d  %d %d ¥n", x[3], x[4], x[5], x[6]);
    printf("%d %d¥n", x[7], x[8]);
    printf("-----¥n");
}

```

```

}

int main(void) {
    int temp; //問題で問われている操作終了後の size を保管する変数
    put(7); output();
    put(3); output();
    put(1); output();
    put(9); output();
    put(3); output();
    printf("get()=%d¥n", get()); output();
    put(4); output();
    put(6); output();
    put(8); output();
    printf("get()=%d¥n", get()); output();
    temp = size;

    heapsort(size);

    printf("x=");
    for (int i = 0; i < temp; i++) { //size の分だけ要素を取り出す
        printf("%d,", x[i]);
    }
    printf("¥n");
    printf("size = %d¥n", size);

    return 0;
}

```